

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Angelo Morzenti

Exam of Tue 18 JANUARY 2022 - Part Theory

WITH SOLUTIONS - FOR TEACHING PURPOSES HERE THE SOLUTIONS ARE WIDELY
COMMENTED

LAST + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

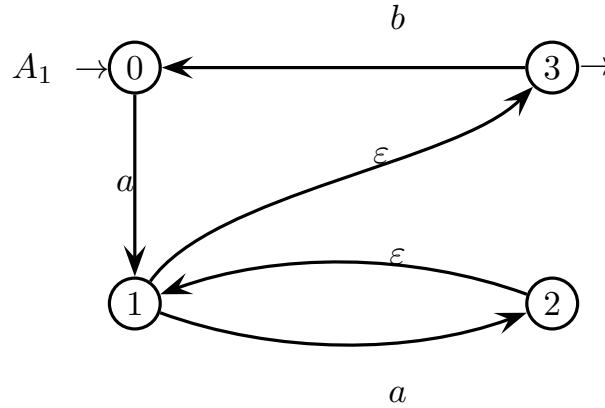
SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.00m

1 Regular Expressions and Finite Automata 20%

1. Consider the nondeterministic finite-state automaton A_1 below, which has spontaneous transitions (ε -transitions), over the two-letter alphabet $\{a, b\}$:



Answer the following questions:

- (a) List all the strings of length 3 or less that are recognized by automaton A_1 .
- (b) Remove the spontaneous transitions (ε -transitions) from automaton A_1 , by using the back-propagation method. Then determinize and minimize the resulting automaton, if necessary (call A_2 the final automaton obtained).
- (c) Determine if the language $L(A_1)$ recognized by automaton A_1 is local or not.
- (d) Consider the regular expression R below, over the two-letter alphabet $\{a, b\}$:

$$R = (a b)^* a$$

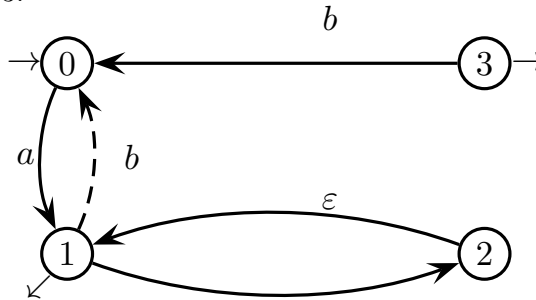
Build a deterministic automaton A_3 that recognizes the language $L(R)$, by using the BS (Berry-Sethi) method.

- (e) Build an automaton A_4 that accepts the difference language $L(A_1) - L(R)$, i.e., the intersection language $L(A_1) \cap \overline{L(R)}$.
- (f) (optional) Find a regular expression R' equivalent to the automaton A_4 built at point (e), by using the BMC (node elimination) method.

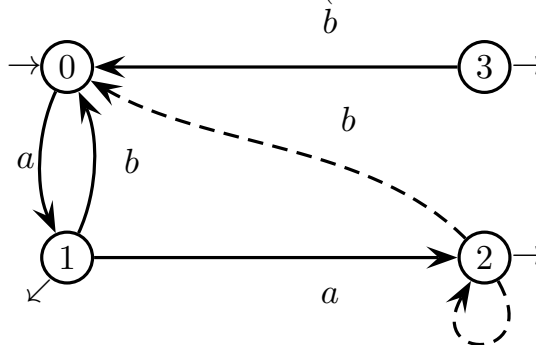
Solution

- (a) Here are all and only the strings of length 3 or less accepted by automaton A_1 :
 a , aa , aaa and aba .
- (b) The deterministic and minimal automaton A_2 is obtained by cutting the ε -transitions through back-propagation, followed by determinization and minimization:

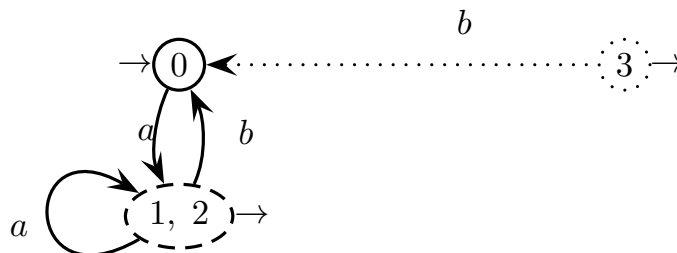
- Remove the arc $1 \xrightarrow{\varepsilon} 3$. Then back-propagate the arc $3 \xrightarrow{b} 0$ and the exit arrow from node 3:



- Remove the arc $2 \xrightarrow{\varepsilon} 1$. Then back-propagate the arc $1 \xrightarrow{a} 2$ (original), the arc $1 \xrightarrow{b} 0$ and the exit arrow from node 1 (both introduced previously):

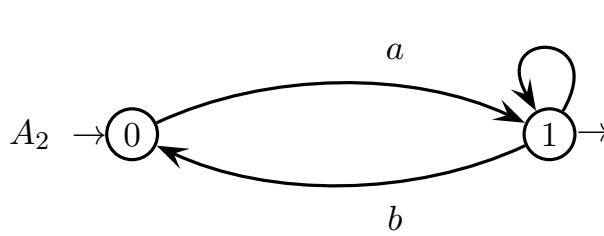


- Now there are no ε -transitions left. Notice that the final state 3 is unreachable, thus cancel it together with the arcs and arrows that touch it (see it dotted below). Since the resulting automaton happens to be already deterministic, it does not need to be determinized, and we can immediately apply minimization theory. Therefore notice that the two final states 1 and 2 are undistinguishable, since both stay inside their group with input a and go to the next state 0 with input b . Thus merge them into one state $(1, 2)$:



The two states left, ie., 0 and $(1, 2)$, are clearly distinguishable, as the former is non-final and the latter is final, thus no more minimization is possible.

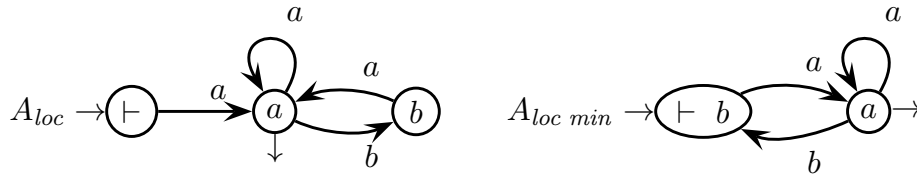
- After cleaning and minimizing, the automaton is (obviously) still deterministic. Thus the final deterministic and minimal automaton A_2 is as below:



Alternatively the removal order can be: first step removes arc $2 \xrightarrow{\varepsilon} 1$, and second step removes arc $1 \xrightarrow{\varepsilon} 3$. Notice that in the first step it is necessary to back-propagate the arc $1 \xrightarrow{\varepsilon} 3$, which action introduces a new arc $2 \xrightarrow{\varepsilon} 3$, to be removed in a third step. The final result (after cleaning and minimizing) is the same.

A slightly different approach (explained in the textbook) first computes the transitive closure of the two (original) ε -transitions (this immediately introduces the new arc $2 \xrightarrow{\varepsilon} 3$), and then removes the three ε -transitions by back-propagation in any order (in this case it is not necessary to back-propagate the arc $1 \xrightarrow{\varepsilon} 3$). The final result (after cleaning and minimizing) is the same.

- (c) The language $L(A_1)$ is local. *Proof 1* – The deterministic automaton $A_2 \equiv A_1$ is normalized local, as each input letter enters exactly one state (minimality is not required). *Proof 2* – The local sets (obtained e.g. from A_2) are first a , last a , and digrams aa , ab and ba ; draw the local automaton A_{loc} for such sets and, after minimizing (merge states $\vdash$ and b), see that A_{loc} becomes identical to $A_2 \equiv A_1$:

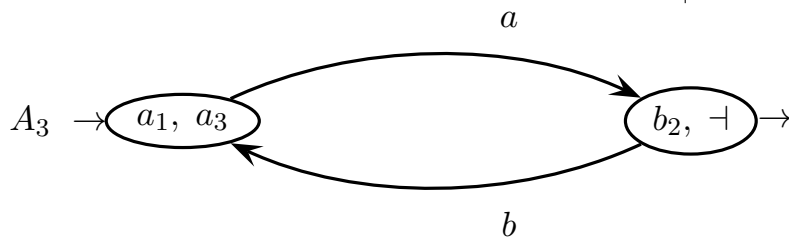


Proof 3 – The language is characterized by the local sets, as you may test with the sample strings of point (a) (proof 3 is less rigorous than proofs 1 and 2).

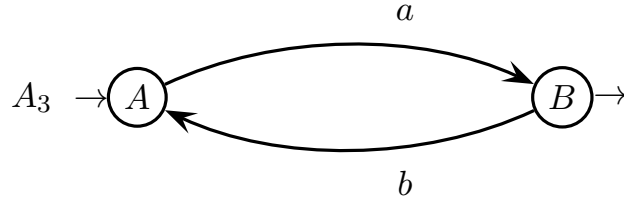
- (d) Here is the application of the BS (Berry-Sethi) method to the regular expression R , to obtain the equivalent deterministic automaton A_3 :

$$R_{\#} = (a_1 b_2)^* a_3 \dashv$$

initial(s)	a_1, a_3
generator	follower(s)
a_1	b_2
b_2	a_1, a_3
a_3	$\dashv$

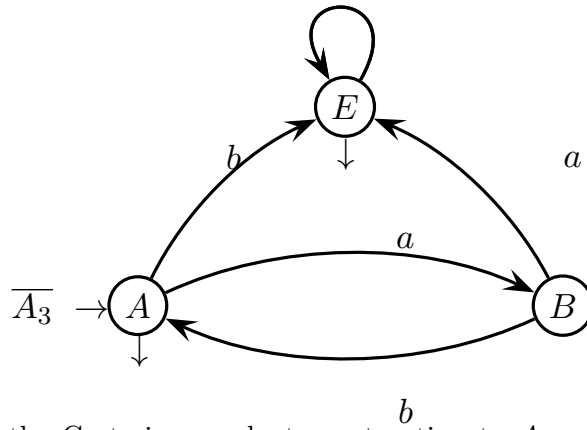


Automaton A_3 is deterministic by construction. Additionally notice that it is minimal, since its two states are distinguishable (non-final and final, respectively). After renaming its states, it becomes:



- (e) To build an automaton A_4 for the difference language $L(A_1) - L(R) = L(A_1) \cap \overline{L(R)}$, first build the automaton that recognizes the complement language $\overline{L(R)}$, of course starting from the deterministic automaton A_3 previously obtained, which is equivalent to R , then intersect automaton A_2 , which is equivalent to A_1 but has fewer states, and the complement automaton $\overline{A_3}$, by means of the Cartesian product of automata. Thus:

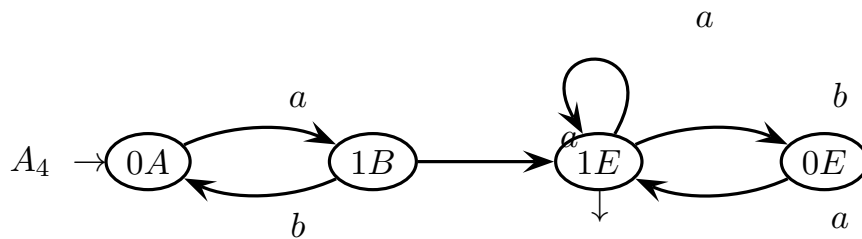
- Obtain the complement automaton $\overline{A_3}$, first by completing A_3 with the (non-final) error state E and then by switching the non-final and final states:



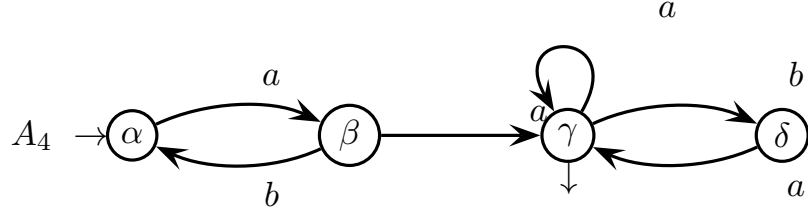
- Now apply the Cartesian product construction to A_2 and $\overline{A_3}$. This is the product state-transition table:

	A	B	E
0	$a: 1B$		$a: 1E$
1	$a: 1B$	$a: 1E$ $b: 0A$	$a: 1E$ $b: 0E$

from which obtain the state-transition graph of automaton A_4 :



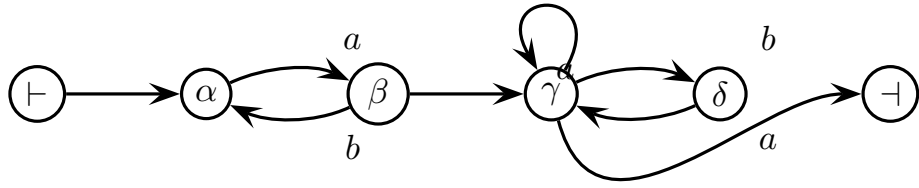
which finally becomes, after renaming the states:



Notice that automaton A_4 is certainly deterministic, since it is the product of two deterministic automata, i.e., A_2 and $\overline{A_3}$. We might additionally verify that it is even minimal, but this is not required.

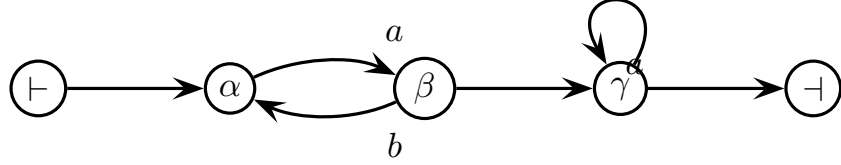
(f) To apply the BMC (node elimination) method and obtain a regular expression R' equivalent to automaton A_4 , possibly do as follows:

- First normalize the automaton so as to have an initial state and a final state that cannot be revisited:



- Then remove node δ and obtain:

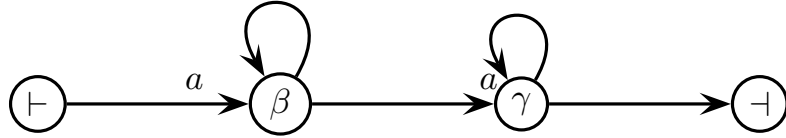
$$a \mid ba$$



- Then remove node α and obtain:

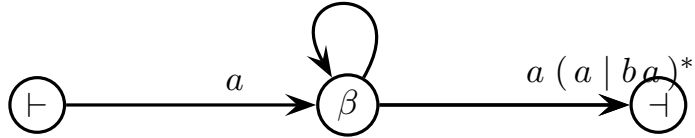
$$ba$$

$$a \mid ba$$

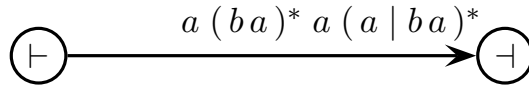


- Then remove node γ and obtain:

$$ba$$



- Finally remove node β and obtain:



Therefore, a possible regular expression R' equivalent to automaton A_4 is:

$$R' = a(ba)^*a(a \mid ba)^*$$

A different elimination order might yield a different (equivalent) version of R' . Some simple algebraic transformations yield these alternative equivalent forms:

$$\begin{aligned}
R' &= a (ba)^* a (a \mid ba)^* \\
&= (ab)^* a a (a \mid ba)^* \\
&= (ab)^* a a ((\varepsilon \mid b) a)^* \\
&= (ab)^* a a ([b] a)^*
\end{aligned}$$

where the square brackets indicate optionality.

2 Free Grammars and Pushdown Automata 20%

1. Consider the four-letter terminal alphabet $\Sigma = \{ a, b, c, d \}$, and the context-free language $L \subseteq \Sigma^*$ defined below:

$$L = \{ a^k b^h c^h d^k \mid k \geq 1 \text{ and } h \geq 0 \}$$

Answer the following questions:

- (a) Write a non-ambiguous BNF grammar G that generates language L , and draw a syntax tree of the valid string:

$$a b b c c d = a b^2 c^2 d$$

- (b) Write a non-ambiguous BNF grammar G' that generates the new language $L' \subseteq L$, in the strings of which every sequence of consecutive letters a is of even length (2, 4, ...) and every sequence of consecutive letters c is of odd length (1, 3, ...). Also draw a syntax tree for the valid string:

$$a a b c d d = a^2 b c d^2$$

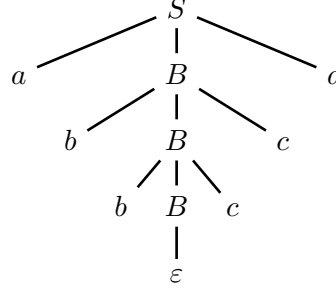
- (c) In the grammar G found at point (a), keep the rule structure and just change letter b into letter a . Is the resulting grammar ambiguous or not? Is the resulting language inherently ambiguous or not? Shortly explain your answers.
- (d) (optional) In the grammar G found at point (a), keep the rule structure and just change letter c into letter a , and change both letters b and d into the empty string ε . Is the resulting grammar ambiguous or not? Is the resulting language inherently ambiguous or not? Shortly explain your answers.
-

Solution

- (a) Grammar G (axiom S), non-ambiguous:

$$G \left\{ \begin{array}{l} S \rightarrow a S d \mid a B d \\ B \rightarrow b B c \mid \varepsilon \end{array} \right.$$

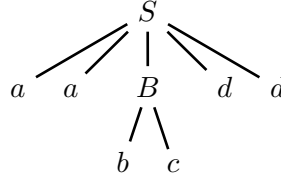
Tree of $a b b c c d = a b^2 c^2 d$:



- (b) Grammar G' (axiom S), non-ambiguous:

$$G' \left\{ \begin{array}{l} S \rightarrow a a S d d \mid a a B d d \\ B \rightarrow b b B c c \mid b c \end{array} \right.$$

Tree of $a a b c d d = a^2 b c d^2$:



- (c) After changing the letters in G as said, i.e., b into a , language L becomes:

$$\left\{ a^l c^h d^k \mid l = k + h \wedge k \geq 1 \wedge h \geq 0 \right\}$$

Here is the resulting grammar (axiom S):

$$\left\{ \begin{array}{l} S \rightarrow a S d \mid a B d \\ B \rightarrow a B c \mid \varepsilon \end{array} \right.$$

The grammar is non-ambiguous because the exponents h and k count the number of rules, and the rules with lhs S have to be applied before the others, i.e., a valid string has exactly one tree. Of course, the language is not inherently ambiguous.

- (d) After changing the letters in G as said, i.e., c into a and b, d into ε , language L becomes $\{ a^l \mid l \geq 1 \} = a^+$. Here is the resulting grammar (axiom S):

$$\left\{ \begin{array}{l} S \rightarrow a S \mid a B \\ B \rightarrow B a \mid \varepsilon \end{array} \right.$$

The grammar is ambiguous because it consists of unilinear rules (either right or left) that grow the list of elements a on both sides. The language is not inherently ambiguous, since it is regular, see above. Alternatively notice that it is a one-letter free language, well known to be regular. A trivial non-ambiguous grammar for it is $S \rightarrow a S \mid a$.

2. We intend to model a multiple selection conditional statement, based on the following example:

```
select x - 3 * y to
    2 : begin                -- branch taken if x - 3 * y equals 2
        y := x - z;
        z := 1 * w          -- no final semicolon in a block
    end;
    5, 7 : z := 2;           -- branch with only one statement
    15 : empty;              -- branch with empty executable part
    13 : begin               -- nested blocks
        ...
        begin
            z := 7
        end
    end;
else : z := w;              -- optional default final branch
end
```

This language fragment exhibits the following features:

- comments, following the `--` sign in a line, are not part of the language and must not be modeled
- the `select` statement includes at least one branch (different from the `else` clause) and may contain one additional final optional `else` clause; all the branches and the final `else` clause are terminated by a semicolon `“;”`
- all the branches and the final `else` clause have an executable part that is either `empty`, or a single instruction, or a statement block
- every block contains a non-empty list of statements, where the semicolon appears only as a separator between consecutive statements; the statement list is enclosed within the keywords `begin` and `end`; blocks may be nested
- identifiers and numerical constants are modeled by the terminals `id` and `num`, respectively
- for simplicity, expressions are to be modeled by the nonterminal $\langle \text{EXPR} \rangle$, not to be further detailed
- assignments instructions have the operator `“:=”`, with a variable identifier on the left and an expression on the right
- the `select` statement can be nested
- each branch is labeled by a list of one or more numerical constants, which are separated by commas `“,”`, and the list is terminated by a colon `“:”` sign

Write an extended grammar (*EBNF*), non-ambiguous, that models the multiple selection statement as described above.

Solution

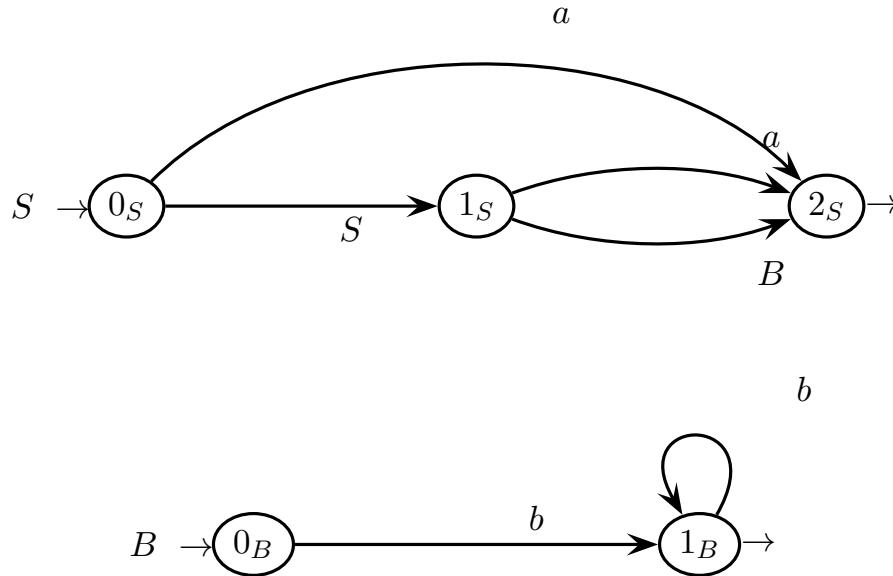
Here below is a possible non-ambiguous grammar (axiom SELECT), of type *EBNF*:

$$\left\{ \begin{array}{l} \langle \text{SELECT} \rangle \rightarrow \text{select } \langle \text{EXPR} \rangle \text{ to } \langle \text{BODY} \rangle \text{ end} \\ \langle \text{BODY} \rangle \rightarrow \langle \text{BRANCH} \rangle^+ [\langle \text{ELSE} \rangle] \\ \langle \text{BRANCH} \rangle \rightarrow \langle \text{LABELS} \rangle \text{ ':' } \langle \text{STTM} \rangle \text{ ';' } \\ \langle \text{LABELS} \rangle \rightarrow \text{num (' , ' num)}^* \\ \langle \text{ELSE} \rangle \rightarrow \text{else } \langle \text{STTM} \rangle \text{ ';' } \\ \langle \text{STTM} \rangle \rightarrow \text{empty} \mid \langle \text{ASGN} \rangle \mid \langle \text{BLOCK} \rangle \mid \langle \text{SELECT} \rangle \\ \langle \text{ASGN} \rangle \rightarrow \text{id ' := ' } \langle \text{EXPR} \rangle \\ \langle \text{BLOCK} \rangle \rightarrow \text{begin } \langle \text{STTM} \rangle \text{ (' ; ' } \langle \text{STTM} \rangle \text{)}^* \text{ end} \end{array} \right.$$

The grammar uses only non-ambiguous constructs, thus reasonably it is non-ambiguous.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar, presented as a machine net, over the two-letter terminal alphabet $\{ a, b \}$ and the nonterminal alphabet $\{ B, S \}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Determine whether the grammar is of type ELR(1), by drawing its complete pilot and examining all the possible conflicts.
- (b) Determine whether the grammar is of type ELL(1), by finding all the guide sets on the call arcs and exit arrows, and examining all the possible conflicts.
- (c) Simulate the ELR recognition algorithm for the valid string below:

$a\ b\ b\ a$

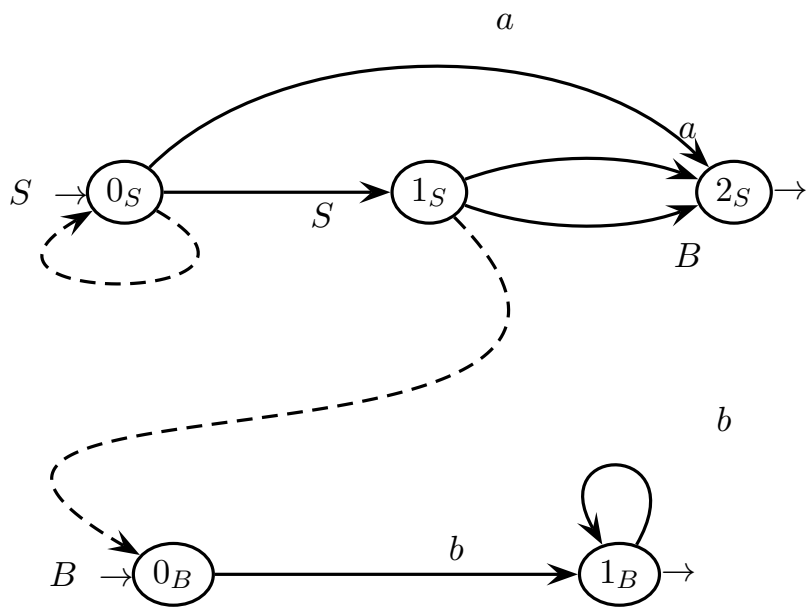
and show the reconstruction of the rightmost derivation. In the case that the grammar is not ELR(1), use the same strategy as Bison does for resolving conflicts: shift/reduce conflicts are resolved by choosing the shift option, and reduce/reduce conflicts (and also convergence conflicts, if any) are resolved by choosing the first rule (for a machine net, the topmost arc).

- (d) (optional) Determine what is the least general language class, to which the language accepted by the machine net belongs.

draw here te complete pilot of the grammar – question (a)

discuss here the possible pilot conflicts – question (a)

machine net to be completed with the guide sets – question (b)



discuss here the possible guide set conflicts – question (b)

ELR simulation table – question (c) – (number of rows not significant)

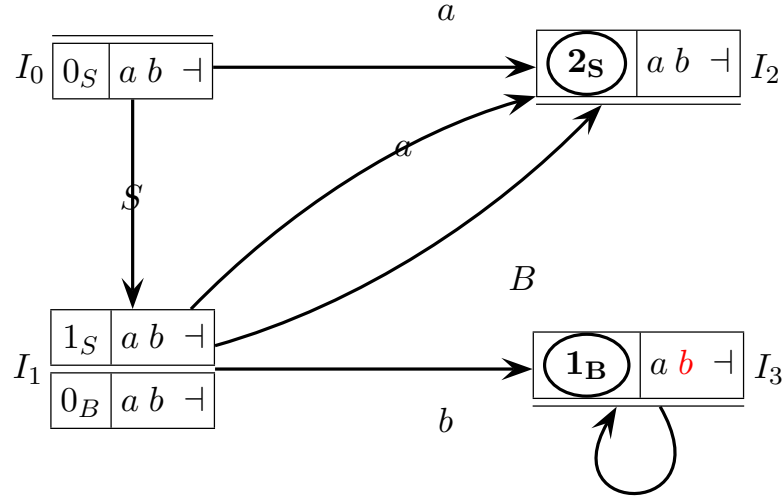
<i>stack of m-states and grammar symbols</i>	<i>input left to be scanned</i>	<i>move executed</i>
I_0	$a\ b\ b\ a\ \dashv$	initial configuration
$I_0\ a\ I_{\dots}$	$b\ b\ a\ \dashv$	terminal shift: $I_0 \xrightarrow{a} I_{\dots}$

write here the reconstruction of the rightmost derivation of the string – question (c)

space for answering question (d)

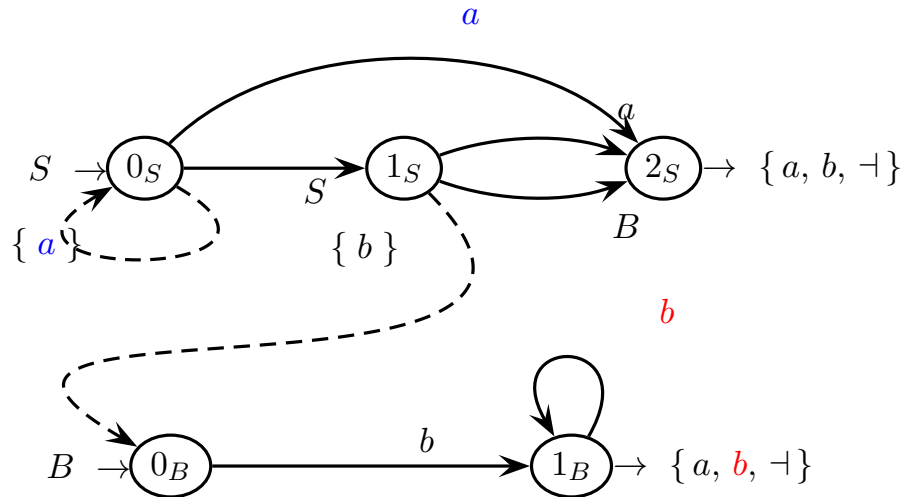
Solution

- (a) Here is the complete pilot automaton of the grammar, with four m-states, where both nonterminals B and S have a look-ahead set that includes all the input letters and the terminator, too:



There is a shift/reduce conflict in the m-state I_3 on the input letter $\textcolor{red}{b}$ (in red). There are no other conflicts. Therefore the grammar is not of type ELR(1).

- (b) Here are the guide sets of the machine net, on the call arcs and exit arrows:



There is a guide set conflict in the net state 0_S on the input letter a (in blue), and another one in the final net state 1_B on the input letter b (in red). There are no other guide set conflicts. Therefore the grammar is not of type ELL(1) either. Of course, at least one guide set conflict was expected, as the grammar is not of type ELR(1), and it even exhibits an immediate leftmost recursion (on nonterminal S). Notice that the conflict in the state 0_S (caused by the leftmost recursion) is not present in the pilot, which can carry on the two derivations altogether, while the conflict in the state 1_B is essentially the shift/reduce one already found in the pilot.

(c) Simulation of ELR recognition algorithm for the valid string $abba$:

<i>stack of altern. m-states and grammar symbols</i>	<i>input left to be scanned</i>	<i>move executed</i>
I_0	$a\ b\ b\ a\ \dashv$	initial configuration
$I_0\ a\ I_2$	$b\ b\ a\ \dashv$	terminal shift: $I_0 \xrightarrow{a} I_2$
I_0	$b\ b\ a\ \dashv$	reduce 1: $a \rightsquigarrow S$
$I_0\ S\ I_1$	$b\ b\ a\ \dashv$	nonterm. shift on S : $I_0 \xrightarrow{S} I_1$
$I_0\ S\ I_1\ b\ I_3$	$b\ a\ \dashv$	terminal shift on b : $I_1 \xrightarrow{b} I_3$
$I_0\ S\ I_1\ b\ I_3\ b\ I_3$	$a\ \dashv$	terminal shift on b : $I_3 \xrightarrow{b} I_3$ the shift/reduce conflict in I_3 on b is solved in favour of shift
$I_0\ S\ I_1$	$a\ \dashv$	reduce 2: $bb \rightsquigarrow B$
$I_0\ S\ I_1\ B\ I_2$	$a\ \dashv$	nonterm. shift on B : $I_1 \xrightarrow{B} I_2$
I_0	$a\ \dashv$	reduce 3: $SB \rightsquigarrow S$
$I_0\ S\ I_1$	$a\ \dashv$	nonterm. shift on S : $I_0 \xrightarrow{S} I_1$
$I_0\ S\ I_1\ a\ I_2$	$\dashv$	terminal shift on a : $I_1 \xrightarrow{a} I_2$
I_0	$\dashv$	reduce 4: $Sa \rightsquigarrow S$
initial configuration	empty tape	reduced to axiom – accept

Here is the reconstruction of the rightmost derivation of the string:

$$\textcolor{red}{S} \xrightarrow[\text{reduce 4}]{S \rightarrow Sa} \textcolor{red}{S}\ a \xrightarrow[\text{reduce 3}]{S \rightarrow SB} S\ \textcolor{red}{B}\ a \xrightarrow[\text{reduce 2}]{B \rightarrow bb} \textcolor{red}{S}\ b\ b\ a \xrightarrow[\text{reduce 1}]{S \rightarrow a} a\ b\ b\ a$$

The list of reductions is scanned backwards (from bottom to top in the table) and each reduction represents a rightmost derivation step. For instance, notice that nonterminal B (in red), when is derived is the rightmost symbol.

- (d) The least general language family to which the language accepted belongs, is the regular one. In fact, the language can be expressed by the (ambiguous) regular expression obtained as below:

$$\begin{aligned} L(B) &= b^+ && // \text{ from machine } M_B \\ L(S) &= a \mid L(S) (a \mid L(B)) && // \text{ from machine } M_S \\ &= a (a \mid L(B))^* && // \text{ by Arden identity} \\ &= a (a \mid b^+)^* && // \text{ by substitution} \end{aligned}$$

Therefore language $L(G) = L(S)$ is regular (and of course it contains infinitely many strings, therefore it does not belong to the family of finite languages).

The RE ambiguity suggests that the grammar may be ambiguous. This is true, and the reader may wish to simulate that taking the reduction move, instead of shifting, also leads to accept the string $abba$, but with a different tree or rightmost derivation:

$$S \xrightarrow{S \rightarrow Sa} S\ a \xrightarrow{S \rightarrow SB} S\ B\ a \xrightarrow{B \rightarrow b} S\ b\ a \xrightarrow{S \rightarrow SB} S\ B\ b\ a \xrightarrow{B \rightarrow b} S\ b\ b\ a \xrightarrow{S \rightarrow a} a\ b\ b\ a$$

4 Language Translation and Semantic Analysis 20%

1. Consider the language of 2-level infix arithmetic expressions, i.e., without brackets, with left-associative addition $+$ and right-associative multiplication $\times$. Precedence is as usual: multiplication $\times$ takes precedence over addition $+$. The operands are schematized by terminal a . Below there is a BNF source grammar G^s for such a language (axiom E):

$$G^s \left\{ \begin{array}{l} E \rightarrow E + T \\ E \rightarrow T \\ T \rightarrow a \times T \\ T \rightarrow a \end{array} \right.$$

Answer the following questions:

- (a) Consider a syntactic translation function τ_1 that puts both addition and multiplication into prefix form. Write the destination (target) grammar G_1^d that computes τ_1 . Write the translations of the two source strings:

$$a + a \times a \qquad a + a \times a \times a + a$$

To verify, draw the source and the destination (target) syntax trees of the former string, and only the destination (target) syntax tree for the latter string.

- (b) Then consider a syntactic translation function τ_2 that puts addition into prefix form and multiplication into postfix form. Write the destination (target) grammar G_2^d that computes τ_2 . Write the translations of the two source strings:

$$a + a \times a \qquad a + a \times a \times a + a$$

To verify, draw the source and the destination (target) syntax trees of the former string, and only the destination (target) syntax tree for the latter string.

- (c) (optional) Finally, consider the inverse translation functions of τ_1 and τ_2 , i.e., switch the source and target grammars. Are such inverse translations deterministic of type $LL(k)$ or $ELL(k)$ for some $k \geq 1$? Explain your answers.

Solution

(a) Here is the target (destination) grammar G_1^d :

$$G_1^d \left\{ \begin{array}{l} E \rightarrow + E T \\ E \rightarrow T \\ T \rightarrow \times a T \\ T \rightarrow a \end{array} \right.$$

Translations (the numbering help us find the correspondence between operators):

$$\tau_1(a + a \times a) = +a \times aa$$

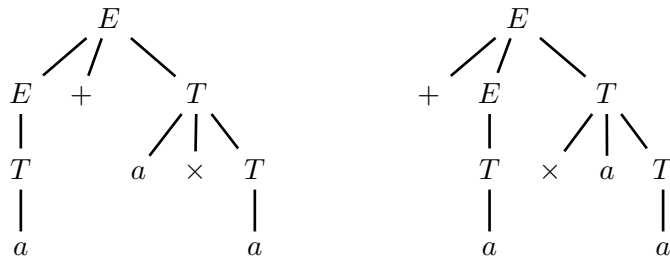
$$\tau_1(a \underset{1}{+} a \underset{1}{\times} a \underset{2}{\times} a \underset{2}{+} a) = \underset{2}{+} \underset{1}{+} a \underset{1}{\times} a \underset{2}{\times} a a a$$

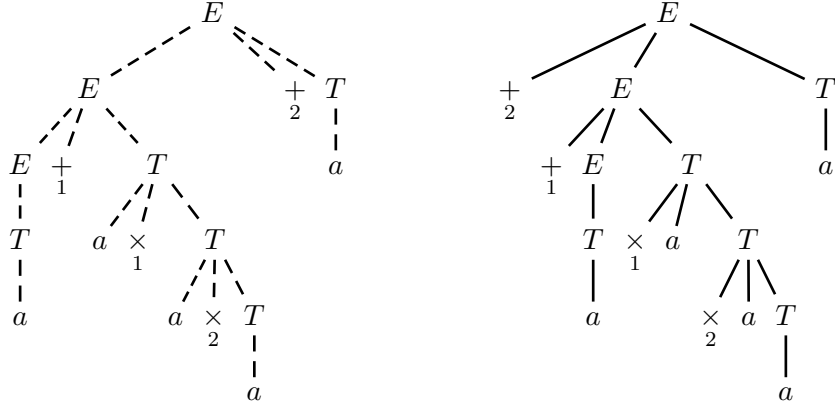
The trees below reasonably prove the correctness of the two sample translations. However, for more safety, consider for instance the second string, and remember that addition and multiplication are left- and right-associative, respectively, which corresponds to the nonterminals E and T being left- and right-recursive in the source grammar G^s , respectively. Thus parenthesize the string accordingly (for better clarity, use square and round brackets for $+$ and $\times$, respectively), to make the association order visible, and stepwise translate the string (for instance starting from the innermost operator):

$$\begin{aligned}
a \overset{+}{\underset{1}{+}} a \overset{\times}{\underset{1}{\times}} a \overset{\times}{\underset{2}{\times}} a \overset{+}{\underset{2}{+}} a &\Rightarrow \left[\left[a \overset{+}{\underset{1}{+}} \left(a \overset{\times}{\underset{1}{\times}} \left(a \overset{\times}{\underset{2}{\times}} a \right) \right) \right] \overset{+}{\underset{2}{+}} a \right] \\
&\Rightarrow \left[\left[a \overset{+}{\underset{1}{+}} \left(a \overset{\times}{\underset{1}{\times}} \overset{\times}{\underset{2}{\times}} (a a) \right) \right] \overset{+}{\underset{2}{+}} a \right] \\
&\Rightarrow \left[\left[a \overset{+}{\underset{1}{+}} \overset{\times}{\underset{1}{\times}} \left(a \overset{\times}{\underset{2}{\times}} (a a) \right) \right] \overset{+}{\underset{2}{+}} a \right] \\
&\Rightarrow \left[\overset{+}{\underset{1}{+}} \left[a \overset{\times}{\underset{1}{\times}} \left(a \overset{\times}{\underset{2}{\times}} (a a) \right) \right] \overset{+}{\underset{2}{+}} a \right] \\
&\Rightarrow \overset{+}{\underset{2}{+}} \left[\overset{+}{\underset{1}{+}} \left[a \overset{\times}{\underset{1}{\times}} \left(a \overset{\times}{\underset{2}{\times}} (a a) \right) \right] a \right] \\
\overset{+}{\underset{2}{+}} \overset{+}{\underset{1}{+}} a \overset{\times}{\underset{1}{\times}} a \overset{\times}{\underset{2}{\times}} a a a
\end{aligned}$$

After removing all the brackets, which are always unnecessary for prefix and postfix forms, the translated string is eventually obtained.

Trees (all of them, including the source tree not required, dashed):





(b) Here is the target (destination) grammar G_2^d :

$$G_2^d \begin{cases} E \rightarrow + E T \\ E \rightarrow T \\ T \rightarrow a T \times \\ T \rightarrow a \end{cases}$$

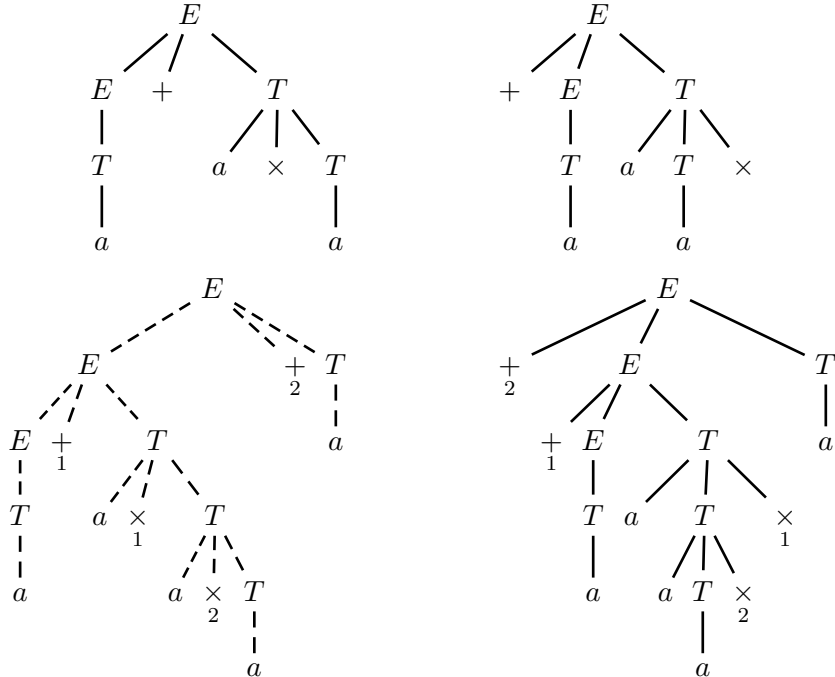
Translations (the numbering help us find the correspondence between operators):

$$\tau_2(a + a \times a) = + a a a \times$$

$$\tau_2(a +_1 a \times_1 a \times_2 a +_2 a) = +_2 +_1 a a a a \times_2 \times_1 a$$

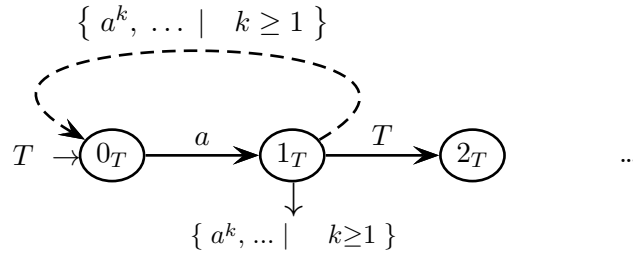
The second string can be translated as done before, just taking care that now multiplication must go into postfix form.

Trees (all of them, including the source tree not required, dashed):



- (c) The target grammar G_1^d (which is the source one for the inverse translation τ_1^{-1}) is clearly of type $\text{LL}(1)$, as all its alternative rules start by a different terminal, i.e., their guide sets are disjoint, therefore translation τ_1^{-1} is deterministic.

The target grammar G_2^d (which is the source one for the inverse translation τ_2^{-1}) is neither $\text{LL}(k)$ nor $\text{ELL}(k)$ for any $k \geq 1$. In fact, its last two alternative rules share string a^k in their guide sets, for all $k \geq 1$. Therefore the grammar is not $\text{LL}(k)$ for any $k \geq 1$. This situation does not change even if such rules are unified as one machine (in which case the bottom rule becomes an exit arrow). To better see this fact, consider the machine net fragment below:



The guide set on the call arc contains a^k for all $k \geq 1$ (plus possibly other strings, not relevant here), as nonterminal T starts by a . The guide set on the exit arrow contains a^k as well, as nonterminal T is at the end of nonterminal E (by rule $E \rightarrow T$), and nonterminal E is followed by nonterminal T (by rule $E \rightarrow +ET$). Therefore translation τ_2^{-1} is not deterministic of type $\text{ELL}(k)$ for any $k \geq 1$.

2. A language is composed of non-empty strings over the two-letter alphabet $\{ a, b \}$. We intend to assign a numerical value to every string of this language. To this end, each symbol is assigned a value equal to its position p in the string (p starts from 1 for the **leftmost** symbol) multiplied by 2 for symbol a and by 3 for symbol b ; then the values assigned to each symbol are added. For instance, the valid string $a a b a$ is assigned the value:

$$v = 1 \times \underset{a}{2} + 2 \times \underset{a}{2} + 3 \times \underset{b}{3} + 4 \times \underset{a}{2} = 23$$

With reference to the syntactic support below (axiom S):

$$\left\{ \begin{array}{l} S \rightarrow L \\ L \rightarrow a L \\ L \rightarrow b L \\ L \rightarrow a \\ L \rightarrow b \end{array} \right.$$

we want to design an attribute grammar G_1 with the **right** attribute p , of type integer, for nonterminal L , which represents the positions in the string, and the **left** attribute v , of type integer, for nonterminals S and L , which represents the value assigned to a string (and to all its suffix strings) according to the above described rule.

Answer the following questions (use the spaces / tables / trees on the next pages):

- Define the semantic rules (functions) of G_1 for computing the attribute values.
- Write the attribute values of all the nodes in the syntax tree of the string:

$b a a$

- Say if the attribute grammar G_1 is of type one-sweep and if it is of type L . Justify your answers by referring to the specific grammar G_1 (avoid generic answers).
- Next imagine that, for the same language and syntactic support, the numerical value for every string is computed the same way, except that the position p starts from 1 for the **rightmost** symbol. For instance, the valid string $a a b a$ is now assigned the value $v = 1 \times \underset{a}{2} + 2 \times \underset{b}{3} + 3 \times \underset{a}{2} + 4 \times \underset{a}{2} = 22$.

With reference to the same syntactic support shown above, we want to design an attribute grammar G_2 with the **left** attribute p , of type integer, for nonterminal L , which represents positions in the string, and the **left** attribute v , of type integer, for nonterminals S and L , which represents the value assigned to a string (and to all its suffix strings) according to this new method.

Define the semantic rules (functions) of G_2 for computing the attribute values, according to this new method. Decorate with the attribute values all the nodes in the syntax tree of the string $b a a$.

table to be filled in for question (a) – grammar G_1

#	<i>syntax</i>	<i>semantic rules for the attributes</i>
---	---------------	--

1:	$S_0 \rightarrow L_1$	
----	-----------------------	--

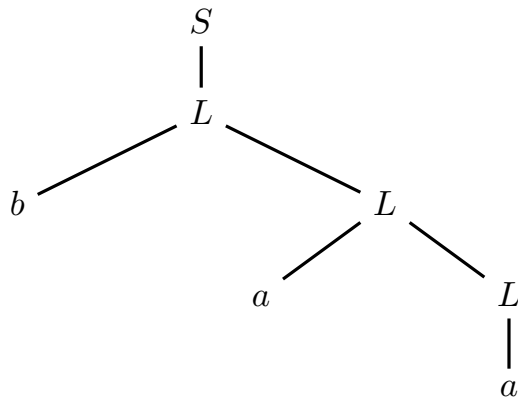
2:	$L_0 \rightarrow a L_1$	
----	-------------------------	--

3:	$L_0 \rightarrow b L_1$	
----	-------------------------	--

4:	$L_0 \rightarrow a$	
----	---------------------	--

5:	$L_0 \rightarrow b$	
----	---------------------	--

tree to be decorated for question (b) – string $b a a$

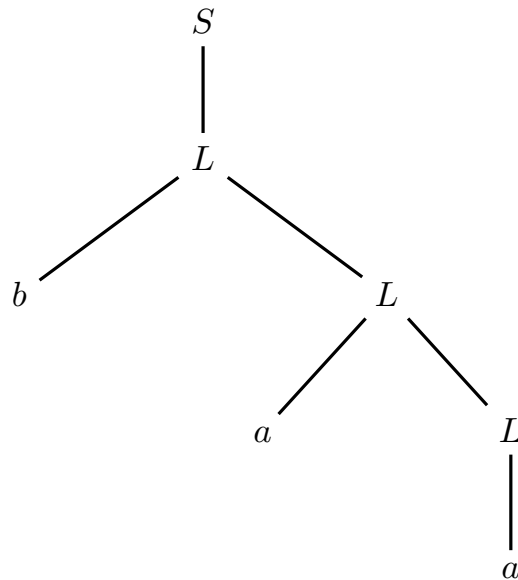


space for answering question (c) – continue elsewhere if necessary

table to be filled in for question (d) – grammar G_2

#	<i>syntax</i>	<i>semantic rules for the attributes</i>
1:	$S_0 \rightarrow L_1$	
2:	$L_0 \rightarrow a L_1$	
3:	$L_0 \rightarrow b L_1$	
4:	$L_0 \rightarrow a$	
5:	$L_0 \rightarrow b$	

tree to be decorated for question (d) – string $b a a$

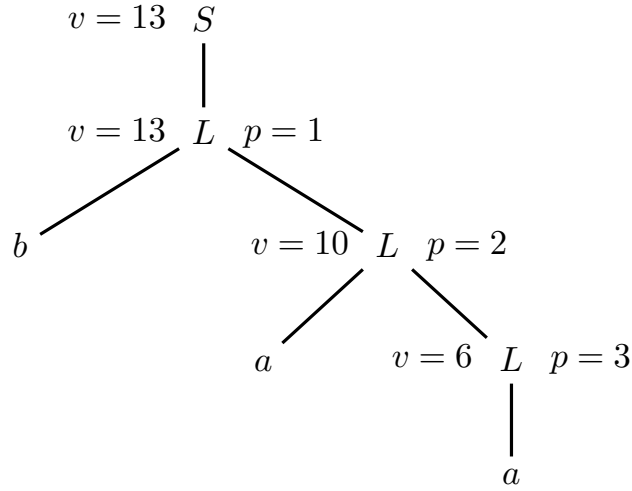


Solution

- (a) Here are the semantic rules for the attribute grammar G_1 :

#	syntax	semantic rules for the attributes
1:	$S_0 \rightarrow L_1$	$p_1 := 1$ $v_0 := v_1$
2:	$L_0 \rightarrow a L_1$	$p_1 := p_0 + 1$ $v_0 := v_1 + 2 \times p_0$
3:	$L_0 \rightarrow b L_1$	$p_1 := p_0 + 1$ $v_0 := v_1 + 3 \times p_0$
4:	$L_0 \rightarrow a$	$v_0 := 2 \times p_0$
5:	$L_0 \rightarrow b$	$v_0 := 3 \times p_0$

- (b) Decorated syntax tree for the string $b a a$ (attributes v and p are left and right):



- (c) In each semantic rule of the attribute grammar G_1 , the right (inherited) attribute p depends at most on its own value in the parent node, and the left (synthesized) attribute v depends at most on its own value in a child node and on the value of p in the same node. Therefore, for the attribute grammar G_1 to be one-sweep, condition 1 (the dependence graph *dep* of the rule is acyclic), condition 2 (no arcs from a left attribute to a right one of any node) and condition 3 (no arc from a left attribute of the parent node to a right one of any child node) are satisfied. Concerning condition 4 (the sibling graph *sibl* of the rule is acyclic), it is trivially satisfied as no syntactic rule has more than one nonterminal in the right part. This proves that the attribute grammar G_1 is one-sweep.

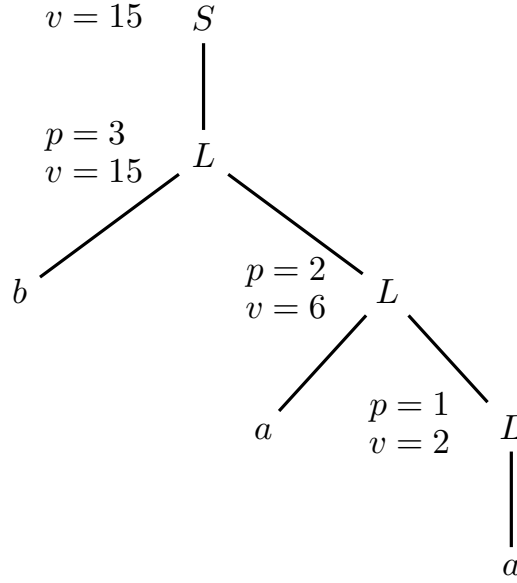
Also the L condition is trivially satisfied as no syntactic rule has more than one nonterminal in the right part, so that the required topological sorting includes only one nonterminal, i.e., it is unnecessary to sort the subtrees.

The reader may wish to (easily) verify that the syntactic support is not of type LL(1), but is of type LL(2). Therefore it would be possible to design an integrated syntactic-semantic analyzer of the recursive descent type.

- (d) Here are the semantic rules for the attribute grammar G_2 , which is based on the new method:

#	syntax	semantic rules for the attributes
1:	$S_0 \rightarrow L_1$	$v_0 := v_1$
2:	$L_0 \rightarrow a L_1$	$p_0 := p_1 + 1$ $v_0 := 2 \times p_0 + v_1$
3:	$L_0 \rightarrow b L_1$	$p_0 := p_1 + 1$ $v_0 := 3 \times p_0 + v_1$
4:	$L_0 \rightarrow a$	$p_0 := 1$ $v_0 := 2$
5:	$L_0 \rightarrow b$	$p_0 := 1$ $v_0 := 3$

Decorated syntax tree for the string $b a a$ (both attributes p and v are left):



All the considerations about attribute grammar G_1 that are listed at point (c), apply to attribute grammar G_2 as well, and even more straightforwardly, since the latter grammar is purely synthesized (it does not have any right attributes).